Plan for the 19-20 Nov finale. Status on 23 Oct 2026: the firmware compiles; nothing has run on hardware yet.

Electronics bench demo: plan for the 19–20 Nov finale

Status: plan written 20 Oct 2026, before any code changed. P0-1 to P0-3 below are now done; see README.md. It is based on a fresh read of every file in `firmware/edge_node`, our internal audit of 18 Oct 2026 (code review, open questions, risk register; not yet published), the cost sheet in `hardware/`, and the DFRobot wiki for the EC probe.

1. What the demo has to show

The deck now promises it (slide 8, backup A8 and A9): **"ESP32 with a real probe: reads, decides, acts (light, pump, alert)"**, plus** "the fixes from our audit: sensor-fault state, saved baseline, one bottle per event"**.

The demo, about 90 seconds on a table:

Step	What you do	What the jury sees	What it proves
1	Probe in a jar of tap water	Beacon green; the screen shows EC (a few hundred µS/cm) and NORMAL	Reads a real industrial probe over RS485
2	Add a pinch of salt (about 0.5 g per glass)	EC jumps above 2,250 µS/cm, the beacon turns red, the pump fills one bottle, and an alert reaches a phone	Hard-limit track, real actuators
3	Leave the salt in	No second bottle	One bottle per event (audit F7)
4	Unplug the probe cable	The beacon shows SENSOR FAULT, never green	Fail-safe sensing (audit F2, the most important fix)
5	Switch the board off and on	It keeps its learned baseline and doesn't start learning again	Saved baseline (audit F5)
Optional	A smaller pinch in fresh tap water	EC rises but stays under 2,250; after 2 of 3 readings, EVENT	The statistical track

Use a pinch, not a spoonful. A spoonful in a glass is about 35,000 µS/cm, which is beyond the probe's 20,000 µS/cm range.

Label it honestly on the screen and when you speak:

- It's a bench demo running in demo mode: one reading every 15 s instead of every 15 min, and 5 minutes of learning instead of 7 days.
- The alert goes over WiFi on the bench; the float is designed to use LTE.

2. Scope I recommend (your decisions)

Decision	Recommendation	Why	Alternative
Which probe	**EC only (DFRobot SEN0707, K=10)**, the probe in the design	Easiest to show (salt), robust (no glass bulb), and it trips the design's own 2,250 µS/cm alarm level	pH (₹16,500) needs buffers and must be stored wet; DO (₹28,900) can't be changed quickly on a table
Beacon	**WS2812B LED ring** under a translucent dome printed on the H2D	One GPIO, very visible, fast to build	PCA9685 + 3 MOSFETs + a 12 V RGB LED (matches the design; more wiring; do it after the finale)

Decision	Recommendation	Why	Alternative
Alert	**WiFi to a phone (Telegram bot)**, through a phone hotspot at the venue	A few lines of code; no SIM or 2 A modem supply	A7670C LTE modem with an SMS driver (the design path; P1 work)
Clock	**DS3231 RTC module** (a design part)	Real time stamps on alerts and bottles (audit F4)	Network time over WiFi only
Screen	**0.96-inch OLED** showing EC, state and bottles	The jury can read the numbers from 2 m	Laptop serial log
Leave out for the finale	Carousel, pinch valve, wiper, stage sensor, solar and battery, deep sleep	Not needed to prove read, decide, act	—

The industrial EC probe is ±1 % of full scale, which is ±200 µS/cm at K=10. That's fine for a salt spike, but coarse for clean river water. Revisit the K=10 choice before the pilot. It doesn't affect the demo.

3. Firmware changes

These map to the audit's P0 list, plus the parts of P1 the demo needs. The bench build goes in a new PlatformIO environment (`[env:bench]`, `-D BENCH_DEMO`), so the field build keeps its own settings.

#	Change	Files	Audit	Size	Needs hardware?
P0-1	**Make it compile:** one data model. Rewrite `microsd_logger` and `telemetry_comm` to use `SensorReadings` and `AnomalyResult`. Define `DEVICE_ID`, `FIRMWARE_VERSION`, `RIVER_STRETCH_ID`. Use the `PIN_SD_*` pins. In `main.cpp`, call methods that exist (`logReading`, `serializePayload` + `transmit`).	main.cpp, microsd_logger., *telemetry_comm.*, config.h	F1	1 day	No
P0-1	**Pin the toolchain:** a fixed `espressif32` 6.x release (Arduino-ESP32 2.0.x). Remove the unused OneWire and DallasTemperature libraries. Add the libraries below, each pinned. Add `[env:bench]`.	platformio.ini	§3.1	½ day	No
P0-2	**Fail-safe sensing:** • return NaN on a failed read instead of 7.42 / 380 / 6.85 / 8.2; • health = the worst of the three reads, and the median uses valid reads only; • check the reply's address, function code and byte count; • flush stale bytes before sending; • wait at least 200 ms between polls; • a per-channel "fitted" mask, so the bench polls only EC and doesn't fault on the missing probes.	sensor_interface.*, config.h	F2, R2, R9	1½–2 days	No (test by leaving the probe out)
P0-2	**Engine:** skip invalid channels, never fold them into a baseline, and add a **SENSOR_FAULT** state with its own beacon pattern and alert.	anomaly_engine.*, main.cpp	F2	(in the above)	No

#	Change	Files	Audit	Size	Needs hardware?
P0-3	**State machine:** • reset risk and history when learning ends; • **one sampler trigger per event**, hard limits included; • lockout counted in cycles, not by the unset clock; • **save baselines, the learning counter and the risk state in NVS** (Preferences); • read `esp_reset_reason()` at boot.	anomaly_engine., *autosampler.*, main.cpp	F5, F7, F11, R5, R6	1½–2 days	No
Demo mode	**Bench timings and baseline seeding:** • 15 s cycle with no deep sleep; 20-cycle learning; purge 3 s, fill 8 s; • seed the baseline from the first readings; • make the EWMA rate configurable (see the note after this table).	config.h, anomaly_engine.cpp, main.cpp	F12	½–1 day	No
P1-8	**Probe commissioning:** bus at **4800 baud**, EC probe at **address 1** (the factory defaults). Read the register map from the probe on the laptop before trusting `0x0000 = µS/cm`. Warm-up per probe set in config.	config.h, sensor_interface.cpp	R9, Q5	½–1 day	**Yes** (probe)
P1-7	**Drivers:** • beacon (WS2812 ring); • pump on GPIO14 (the wiring adds a gate pull-down and a flyback diode); • DS3231 time; • OLED status screen.	new beacon/rtc/display modules, autosampler.cpp	F3, F4, R10	1½ days	Yes (modules)
Alert	**WiFi + Telegram** (HTTPS POST). The bot token lives in an untracked `secrets.h` that git ignores and that is never committed. Alerts are rate-limited.	new alert module, .gitignore	F3, R12	1 day	Yes (board + phone)

Why the seeding change is needed: without it, 20 updates at the design's α = 1/96 move the EC baseline only about 19 % of the way from its default (450 µS/cm) to your tap water. The demo would then raise false alarms.

Total: about 10–13 working days of firmware work. I can write and compile all of it once PlatformIO is installed; only the last three rows need the parts on a table.

Libraries to add (pin each to the version that builds):

- Adafruit NeoPixel (beacon ring);
- RTClib (DS3231);
- Adafruit SSD1306 and Adafruit GFX (screen);
- ArduinoJson 6 (already used).

The Telegram alert uses the core's own `WiFiClientSecure` and `HTTPClient`.

How each fix is tested. This PC has no host C/C++ compiler, so tests run **on the ESP32** with `pio test` (Unity). The audit's Python replay of the firmware engine (not yet published) checks the decisions match. The bench checks are the demo's own steps:

- probe unplugged → SENSOR FAULT;
- salt held in → exactly one bottle;
- power cut → baseline kept;
- tap water for 10 minutes → no alarm.

Not needed for the demo:

- P0-4, the real schematic and PCB. The bench uses modules on a board. Its rules still apply to the wiring: common ground, gate pull-down, flyback diode.
- P0-5 and P0-6 are documentation fixes.

4. PlatformIO setup on this PC

PlatformIO and the ESP32 toolchain aren't installed; the audit's setup notes say the first build downloads several hundred MB. **Each download needs your OK.**

1. Install **VS Code**, then the **PlatformIO IDE** extension from VS Code's Extensions panel. PlatformIO installs its own Python environment. If its installer refuses Python 3.14, install Python 3.12 alongside and let PlatformIO use that.
2. Open the folder `firmware/edge_node`. The first build downloads the `espressif32` platform and toolchain.
3. Install the USB driver for the board's USB-serial chip: **CP210x** (Silicon Labs) or **CH340** (WCH). The chip's name is printed on the board.
4. Day-to-day commands, run in `firmware/edge_node`:

```
pio run -e bench
```

```
pio run -e bench -t upload
```

```
pio device monitor -b 115200
```

```
pio test -e bench
```

1. Build output (`.pio/`) and `.vscode/` are already in `.gitignore`.
2. Later, and a separate decision because it adds files to the repository: a GitHub Actions build on every push (the audit's P0-1 "CI build").

Code changes go on a new branch (for example `bench-demo`). Nothing gets committed or pushed without your review.

5. Parts to buy

Prices marked *cost sheet* come from `hardware/AquaSentinel_Cost_Estimate_BOM.csv`: v2 catalogue estimates, including GST. **All other prices are rough guesses at Indian online prices. Check current prices before ordering.** Use anything the university lab already has.

#	Part	Qty	Rough ₹	Notes
1	**DFRobot SEN0707 RS485 EC probe (K=10)**	1	**18,200** *cost sheet*	Supply 10–30 V DC, 0.4 W; RS485 Modbus-RTU, address 1, 4800 baud by default (DFRobot wiki). **Order first: it's the long-lead item.**
2	ESP32-WROOM-32 development board (DevKitC type)	2	500–800 each	One spare for the venue
3	RS485 transceiver module, **3.3 V** (SP3485 or MAX3485), with DE/RE pins	2	100–250 each	The firmware drives DE/RE on GPIO25. Avoid 5 V MAX485 boards
4	USB-to-RS485 adapter	1	250–500	To read the probe from the laptop before wiring the ESP32
5	12 V, 2 A DC adapter + DC-jack-to-screw-terminal	1	300–500	Powers the probe and the pump

#	Part	Qty	Rough ₹	Notes
6	12 V to 5 V buck module (2 A or more)	1	100–250	Runs the ESP32 and the LED ring off 12 V
7	12 V peristaltic pump + silicone tubing	1	800–2,400	The cost sheet's G-01 is ₹2,400 (500 mL/min); a hobby dosing pump is fine on the bench
8	Logic-level N-MOSFET (IRLZ44N, or an AOD4184 module), 1N5819 diode, 100 kΩ and 100 Ω resistors	2 sets	100–200	The pump switch. **Not an IRF520 module**: it needs about 10 V on the gate and won't switch fully from 3.3 V
9	WS2812B LED ring, 12–16 LEDs, + 330 Ω resistor and 1,000 µF capacitor	1	200–400	The beacon; print a translucent dome for it
10	DS3231 RTC module with coin cell	1	150–300	Real time stamps
11	0.96-inch SSD1306 I2C OLED	1	200–350	Status screen for the jury
12	microSD module + 8–16 GB card	1	250–450	Optional: the event log
13	Breadboard or perfboard, jumper wires, screw terminals, heat-shrink, a tray or small box	—	500–900	Keep the electronics above the water
14	EC calibration solution, 1,413 µS/cm	1	300–700	Check the probe before every demo
15	Two clear jars, a sample bottle, salt, distilled water, a drip tray	—	200–500	
	Total		**≈ 23,000–27,000**	About ₹5,000–9,000 without the probe

Tools: a multimeter and a soldering iron.

If the probe isn't on the table by 8 Nov, show the same read, decide, act chain with a low-cost analog TDS module. Label it clearly: it isn't the industrial probe in the design. The sensor-fault and saved-baseline fixes can be shown either way.

6. Bench wiring

The pin numbers are the ones in `include/config.h`.

From	To	Notes
12 V +	Probe **brown** (VCC), pump +, buck module input	Wire colours from the DFRobot wiki
GND	Probe **black**, MOSFET source, buck GND, ESP32 GND, RS485 GND	**One common ground**
Probe **yellow** (A) / **blue** (B)	RS485 module A / B	
RS485 RO / DI / DE+RE	GPIO26 / GPIO27 / GPIO25	UART1 (`PIN_RS485_*`)
Pump −	MOSFET drain	Flyback diode across the pump (stripe to +)
GPIO14	MOSFET gate, through 100 Ω	**100 kΩ from gate to GND**: GPIO14 toggles at boot, and without the pull-down the pump twitches (audit R10)
Buck 5 V	ESP32 5V/VIN pin, LED ring 5 V	1,000 µF across the ring's supply
A free GPIO set in the bench config (for example GPIO4, the modem key pin, which is unused on the bench)	Ring data in, through 330 Ω	

From	To	Notes
GPIO21 / GPIO22	SDA / SCL of the DS3231 and the OLED (3.3 V)	The design's I2C bus

On the bench the probe is powered all the time. The switched sensor rail (GPIO32) and its polarity question (audit F6) wait for the real board.

Safety:

- Use a 12 V DC adapter only; nobody on the team wires mains.
- Keep the electronics on a raised tray, with a drip tray under the jars. Salt water conducts, so keep it off the boards.
- Rinse the probe in clean water after each salt step.

7. Schedule (today to the finale)

When	Work	Who
By 25 Oct (semi-final week)	Approve this plan. **Order the probe and the parts.** Install PlatformIO. I make it compile (P0-1)	You approve and order; I write the code
26 Oct – 1 Nov	P0-2, P0-3, demo mode. On-board tests on a bare ESP32 (the fault path needs no probe)	I write and compile; the software lead flashes and runs the tests
2 – 8 Nov	Probe arrives. Read it on the laptop, then wire it to the ESP32. Beacon, pump and alert	Electronics lead wires; I fix what the tests show
9 – 15 Nov	Full rehearsals of the 90-second demo. The 1:5 model water check the same week. **Record a backup video of the demo**	The team
16 – 18 Nov	Freeze the firmware (tag a version). Pack the spare board, spare cables and the calibration solution. Test the phone hotspot	The team
19 – 20 Nov	Finale	

The main risk is the probe's delivery time. Order it this week.

8. What the demo proves, and what it doesn't

Proves:

- the decision logic runs on the ESP32 with a real industrial probe;
- the audit's three safety fixes work on hardware;
- a reading really turns into a light, a pump run and an alert.

Doesn't prove:

- field robustness or the power budget;
- the LTE path;
- the other three probes, or probe fouling;
- anything about the float, which the 1:5 model and the later tank test cover.

9. What I need from you

1. **Scope:** EC only, LED ring, WiFi alert, DS3231, OLED (section 2), or your changes.
2. **Purchase:** about ₹25,000, ₹18,200 of it the probe. Who orders, and from which supplier.
3. **Install:** OK to install VS Code and PlatformIO on this PC (downloads of several hundred MB).

4. **Code:** after 1–3, I start P0-1 on a new branch and show you the diff before anything is committed.

Sources: firmware files as of commit 840f1a6; the internal audit of 18 Oct 2026 (F1–F18, R2–R12, P0/P1 list); `hardware/AquaSentinel_Cost_Estimate_BOM.csv` (lines E-02, G-01); DFRobot SEN0707 wiki (supply, defaults, wire colours, polling interval, range, accuracy).